Serial Python Minimalis:
Tutorial #09-Larik 2D dengan NumPy

Ahmad R. T. Nugraha

ver. 21 April 2026

Larik (*array*) NumPy yang kita pelajari pada bagian sebelumnya semuanya merupakan pengganti dari `list` Python biasa. Namun, di dalam Python, kita juga bisa menggunakan `list` di dalam `list` (*lists-of-lists*), sebagai contoh:

```
LL = [[11,12,13,14], [15,16,17,18], [19,20,21,22]]
```

Secara alamiah, NumPy turut mendukung penggunaan fungsi `array()` agar kita dapat menghasilkan sebuah larik yang berisi elemen-elemen dari `LL`:

```
A = np.array(LL)
A
```

Output:

```
array([[11, 12, 13, 14],
       [15, 16, 17, 18],
       [19, 20, 21, 22]])
```

Contoh di atas menunjukkan *interpreter* Python memahami `A` bukanlah sebuah `list` biasa, melainkan sebuah objek larik. Menariknya, entitas `A` ini langsung ditampilkan dalam tiga baris, sehingga memudahkan kita untuk melihatnya sebagai entitas **dua dimensi** (2D), mirip dengan matriks dalam matematika. Penggunaan fungsi `print()` untuk mencetak `A` akan memperjelas status `A`:

```
print(A)
```

Output:

```
[[11 12 13 14]
 [15 16 17 18]
 [19 20 21 22]]
```

Bandingkan dengan cara Python biasa menampilkan sebuah *list of lists*:

```
print(LL)
```

Output:

```
[[11, 12, 13, 14], [15, 16, 17, 18], [19, 20, 21, 22]]
```

Selain memisahkan ketiga baris tersebut ke baris yang berbeda, hasil cetakan dari sebuah array tidak lagi menampilkan tanda koma, sehingga memungkinkan kita untuk lebih fokus pada angka-angkanya saja.

1 Pendefinisian Larik 2D

Banyak hal yang telah kita pelajari mengenai array 1D turut berlaku untuk kasus 2D. Misalnya, penggunaan `type()` untuk memeriksa tipe larik atau tipe elemennya. Kita juga dapat membuat larik secara spesifik dengan tipe yang diinginkan.

```
B = np.array(LL,dtype=np.float64)
print(B)
```

Output:

```
[[11. 12. 13. 14.]
 [15. 16. 17. 18.]
 [19. 20. 21. 22.]]
```

```
B.dtype
```

Output:

```
dtype('float64')
```

Di sini, kita secara eksplisit memastikan bahwa kita sedang membuat larik berisi bilangan pecahan (*floats*). Selain itu, perhatikan bahwa semua larik memiliki atribut bernama `dtype` yang merujuk pada tipe data elemen-elemennya Kita juga pernah menggunakan atribut `size` pada larik 1D untuk melihat ada berapa banyak elemen di dalamnya. Atribut ini bekerja dengan sama baiknya pada kasus 2D:

```
A.size
```

Output:

```
12
```

Perhatikan bahwa atribut ini menghitung jumlah total elemen, yang mencakup baris maupun kolom.

Atribut lain yang belum kita bahas sebelumnya adalah `ndim`, yang memberi tahu kita dimensi dari larik NumPy tersebut:

```
A.ndim
```

Output:

```
2
```

Jumlah dimensi dapat dibayangkan sebagai jumlah "sumbu" (*axes*) berbeda yang kita gunakan untuk menyusun elemen. Terminologi ini mungkin agak membingungkan bagi mereka yang memiliki latar belakang aljabar linear. Dalam aljabar linear, kita mengatakan bahwa sebuah matriks A dengan m baris dan n kolom memiliki "dimensi" m dan n, atau terkadang disebut berdimensi $m \times n$. Berbeda dengan hal tersebut, konvensi di dalam NumPy adalah menyebut array seperti `A` memiliki "dimensi dua", karena larik tersebut tersusun dari baris dan kolom (yaitu, tidak masalah ada berapa banyak elemen spesifik di masing-masing baris maupun kolomnya). Dalam contoh `A` kita di atas, dimensinya adalah 2, meskipun di sana terdapat 3 baris dan 4 kolom.

Pertanyaan logis selanjutnya adalah apakah kita bisa mengakses jumlah baris dan kolomnya secara khusus (ingat: atribut `size` memuat hasil kali dari keduanya). Jawabannya, tentu saja, ya. Terdapat atribut lain, yaitu `shape`, yang mengembalikan sebuah *tuple* berisi jumlah elemen pada setiap dimensinya. Sebagai contoh:

```
A.shape
```

Output:

```
(3, 4)
```

Perhatikan bahwa $3 \times 4 = 12$. Seperti yang seharusnya segera kita periksa sendiri, sebuah larik 1D akan mengembalikan nilai 1 untuk `ndim`. Demikian pula, atribut `shape`-nya akan berupa sebuah *tuple* dengan satu elemen, misalnya `(7,)`, yang angkanya untuk kasus 1D itu bernilai sama dengan ukuran total larik. Atribut yang akan paling banyak kita gunakan pada pembahasan selanjutnya adalah `shape`.

Sejauh ini kita baru melihat satu cara untuk membuat larik NumPy 2D, yaitu dengan meneruskan *list of lists* ke fungsi `array()`. Namun, pada dasarnya semua fungsi yang sebelumnya kita gunakan untuk larik 1D juga berfungsi untuk larik 2D. Misalnya, kita bisa menggunakan `zeros()` untuk membuat larik 2D yang penuh dengan angka nol. Alih-alih meneruskan satu angka, kali ini kita akan meneruskan sebuah *tuple*, yang merepresentasikan bentuk (*shape*) dari larik baru yang diinginkan:

```
np.zeros((2,3))
```

Output:

```
array([[0., 0., 0.],
       [0., 0., 0.]])
```

Perlu dicermati baik-baik, kita menggunakan sepasang tanda kurung biasa untuk menandakan fungsi `zeros()` dan sepasang tanda kurung biasa tambahan untuk *tuple* yang memuat bentuk *shape*-nya.

Dengan cara serupa, kita bisa menggunakan `ones()` untuk membuat larik 2D yang penuh dengan angka satu. Kali ini, mari kita terapkan untuk membuat matriks persegi:

```
np.ones((3,3))
```

Output:

```
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])
```

Sekali lagi, kita meneruskan sebuah *tuple* untuk merepresentasikan `shape`.

Dalam aljabar linear, terdapat sebuah matriks yang sangat penting yang terdiri dari angka nol dan satu dalam kombinasi khusus, yakni matriks identitas (*identity*), yang menurut definisinya merupakan matriks persegi. Fungsi NumPy bernama `identity()` dapat menghasilkan tepat apa yang kita inginkan ini:

```
np.identity(3)
```

Output:

```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

Kita bisa sadari bahwa `identity()` hanya mengambil satu argumen tunggal, karena fungsi tersebut memang dirancang untuk menghasilkan matriks persegi. Serupa dengan itu, fungsi-fungsi seperti `zeros()` atau `ones()` sebenarnya bisa menghasilkan larik tiga dimensi, empat dimensi, dan seterusnya, meskipun kita belum akan membutuhkannya. Perhatikan juga bahwa secara bawaan, semua fungsi ini menghasilkan bilangan pecahan (*floats*).

Mari kita kembali sejenak pada kasus operan parameter `shape` saat mencoba membuat sebuah fungsi larik baru. Kita mungkin menyadari bahwa pendefinisian sebuah *list of lists* di Python, seperti pada variabel `LL` kita di atas, agak "kaku dan merepotkan" karena kita memerlukan kurung siku ganda serta beberapa tanda koma yang disisipkan di tempat-tempat yang tepat. Faktanya, seperti yang akan kita lihat di bagian berikutnya, upaya menghindari notasi yang tidak praktis ini merupakan salah satu dari sekian banyak manfaat menggunakan larik NumPy 2D.

Pada titik ini, tidak mengherankan lagi jika NumPy memberi kita pilihan untuk menghindari penggunaan *list of lists* sejak awal. Caranya adalah dengan memulai dari array 1D, yang kemudian dibentuk ulang (*reshaped*) menjadi array 2D melalui fungsi `reshape()`:

```
B = np.array([1,4,9,7,2,8,5,6])
B.shape
```

Output:

```
(8,)
```

```
B = B.reshape((2,4))
B.shape
```

Output:

```
(2, 4)
```

```
B
```

Output:

```
array([[1, 4, 9, 7],
       [2, 8, 5, 6]])
```

Awalnya, `B` adalah sebuah larik 1D (yang kita buat menggunakan `list` Python biasa), terlihat jelas dari nilai pada atribut `shape`-nya. Kita kemudian menerapkan metode `reshape()` pada larik 1D tersebut, sehingga menciptakan sebuah larik 2D (yang disimpan kembali ke dalam variabel `B`). Sekali lagi, nilai atribut `shape` menunjukkan bahwa kondisinya sekarang telah berubah.

Perhatikan bahwa kita bahkan tidak perlu memulainya dari `list` Python. Sebaliknya, contoh `A` yang kita diskusikan di atas dapat ditulis ulang menggunakan `arange()` yang menghasilkan sebuah larik 1D:

```
A = np.arange(11,23)
A = A.reshape((3,4))
A
```

Output:

```
array([[11, 12, 13, 14],
       [15, 16, 17, 18],
       [19, 20, 21, 22]])
```

Larik `A` ini bermula sebagai larik 1D dan kemudian dibentuk ulang menjadi larik 2D dengan meneruskan argumen `shape` yang sesuai. Setelah kita semakin terbiasa dengan larik, kita akan merasa lebih nyaman menggabungkan kedua langkah tersebut di dalam satu baris kode:

```
A = np.arange(11,23).reshape(3,4)
A
```

Output:

```
array([[11, 12, 13, 14],
       [15, 16, 17, 18],
       [19, 20, 21, 22]])
```

Perhatikan betapa mudahnya kita menghindari rentetan tanda kurung siku dan koma yang sebelumnya diperlukan saat mendefinisikan `LL`. Sebagai catatan tambahan, kita sebenarnya bisa membentuk ulang larik cukup dengan mengubah nilai *tuple* pada atribut `shape`, alih-alih menggunakan fungsi `reshape()`).

2 Pengindeksan dan Pemotongan

Kini saatnya untuk melihat salah satu fitur paling menyenangkan dari penggunaan larik 2D di NumPy yang ini kebetulan menjadi nilai jual utamanya, yaitu cara yang intuitif dalam mengakses elemen. Mari kita mulai dari contoh *list of lists* kita:

```
LL = [[11,12,13,14], [15,16,17,18], [19,20,21,22]]
```

Seperti yang mungkin kita ingat, jika kita tertarik pada elemen yang terletak di baris ke-2 dan kolom ke-1, kita akan mengeksekusi:

```
LL[2][1]
```

Output:

```
20
```

Ingatlah bahwa kita menggunakan indeks yang dimulai dari angka 0 ala Python, sehingga baris-barisnya diberi nomor ke-0, ke-1, ke-2, dan analogi yang sama berlaku untuk kolomnya.

Dalam percakapan sehari-hari, contoh di atas merujuk pada baris ketiga dan kolom kedua, tetapi kita akan menggunakan angka, misalnya ke-2, saat menggunakan konvensi indeks mulai 0. Pasangan tanda kurung ganda yang memisahkan angka-angka ini satu sama lain cukup merepotkan dan agak berbeda dari bagaimana notasi matriks umumnya ditulis, misalnya A_{ij} atau $A_{i,j}$. Oleh karena itu, sangat melegakan melihat bahwa elemen array NumPy dapat diindeks cukup dengan menggunakan sepasang tanda kurung siku saja beserta dua angka yang dipisahkan oleh tanda koma, contohnya:

```
A = np.arange(11,23).reshape(3,4)
A
```

Output:

```
array([[11, 12, 13, 14],
       [15, 16, 17, 18],
       [19, 20, 21, 22]])
```

```
A[2,1]
```

Output:

```
20
```

Faktanya, notasi `list` Python yang merepotkan tersebut tetap bekerja pada larik NumPy, namun kita tidak akan menggunakannya dan memilih alternatif yang lebih "bersih" yaitu dua nilai yang dipisahkan koma. Demikian pula, meskipun kita dapat membuat array 2D menggunakan *list of lists*, kita umumnya akan lebih memilih menggunakan array 1D yang diikuti dengan pemanggilan fungsi `reshape()`.

Sekarang kita beralih ke *slicing* (pemotongan). Kita mulai dari kasus yang paling sederhana, yaitu memilih baris tertentu (melalui pengindeksan) dan kemudian mengambil irisan dari beberapa kolom berbeda. Contohnya:

```
A[1,1:3]
```

Output:

```
array([16, 17])
```

Demikian pula, jika kita secara spesifik memilih sebuah baris dan menggunakan irisan segala (*everything-slice*) untuk kolomnya, kita akan mendapatkan baris tersebut seutuhnya:

```
A[1,:]
```

Output:

```
array([15, 16, 17, 18])
```

Menggunakan logika yang sama persis, jika kita memilih kolom tertentu dan menggunakan *everything-slice* untuk barisnya, kita akan mendapatkan kolom tersebut seutuhnya:

```
A[:,2]
```

Output:

```
array([13, 17, 21])
```

Kita bisa terus menumpuk contoh-contoh seperti ini dengan memilih sepasang baris, sepasang kolom, dan lain sebagainya.

Sebagai ringkasan, saat kita menggunakan dua angka untuk melakukan indeks (seperti `A[2,1]`), kita bergerak dari sebuah larik 2D menjadi satu buah angka tunggal. Saat kita menggunakan satu angka untuk mengindeks/*slice* (seperti `A[:,2]`), kita bergerak dari larik 2D menjadi larik 1D. Saat kita menggunakan *slice* penuh (seperti `A[::2,:]`), kita akan mendapatkan himpunan kumpulan baris, kolom, elemen individual, dsb.

Kita dapat menggabungkan apa yang baru saja dipelajari mengenai *slicing* larik 2D dengan apa yang sudah kita pahami tentang *broadcasting* di NumPy. Pertama, pilih subhimpunan (*subset*) tertentu dari larik kita dan tugaskan ke variabel baru:

```
B = A[:2,1:]
B
```

Output:

```
array([[12, 13, 14],
       [16, 17, 18]])
```

Manfaatkan *broadcasting* nilai tunggal maupun *everything-slice* (dua kali berturut-turut) untuk mengatur seluruh elemen dari entitas baru ini menjadi angka nol:

```
B[:,:]  = 0
B
```

Output:

```
array([[0, 0, 0],
       [0, 0, 0]])
```

Pada titik ini, kita perlu mengingat bahwa *everything-slices* di NumPy (walaupun memudahkan kita memasukkan nilai ke setiap elemen secara bersamaan seperti di atas) tidak akan menciptakan salinan. Seperti yang telah kita lihat pada larik 1D, irisan larik merupakan tampilan (*views*) ke larik aslinya. Akibatnya, dengan memodifikasi `B`, kita secara riil juga telah memodifikasi `A`:

```
A
```

Output:

```
array([[11,  0,  0,  0],
       [15,  0,  0,  0],
       [19, 20, 21, 22]])
```

Seperti sebelumnya, jika perilaku seperti ini tidak diinginkan, kita harus menggunakan `numpy.copy()`:

```
A = np.arange(11,23).reshape(3,4)
A
```

Output:

```
array([[11, 12, 13, 14],
       [15, 16, 17, 18],
       [19, 20, 21, 22]])
```

```
B = np.copy(A[:2,1:])
B[:,:]  = 0
B
```

Output:

```
array([[0, 0, 0],
       [0, 0, 0]])
```

```
A
```

Output:

```
array([[11, 12, 13, 14],
       [15, 16, 17, 18],
       [19, 20, 21, 22]])
```

3 Vektorisasi

Sama seperti larik 1D, operasi untuk larik 2D ditafsirkan elemen demi elemen (*elementwise*). Operasi yang tervektorisasi semacam ini memungkinkan kita menulis kode yang sangat rapi dan ringkas, contohnya untuk menjumlahkan dua matriks persegi:

```
A = np.arange(1,10).reshape(3,3)
B = np.arange(11,20).reshape(3,3)
A
```

Output:

```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

```
B
```

Output:

```
array([[11, 12, 13],
       [14, 15, 16],
       [17, 18, 19]])
```

```
A+B
```

Output:

```
array([[12, 14, 16],
       [18, 20, 22],
```

```
        [24, 26, 28]])
```

Hasil ini merupakan perilaku yang memudahkan dan diharapkan.

Di sisi lain, operasi perkalian sederhana juga dijalankan secara elemen demi elemen, yang artinya setiap elemen di `A` dikalikan dengan elemen yang bersesuaian dengan posisinya di `B`:

```
A*B
```

Output:

```
array([[ 11,  24,  39],
       [ 56,  75,  96],
       [119, 144, 171]])
```

Hati-hati hasil ini bisa menjadi perilaku tak terduga jika kita sebenarnya sedang bermaksud melakukan perkalian matriks. Sekali lagi, operasi larik selalu dijalankan elemen demi elemen, sehingga produk hasil kali antara dua buah larik dua dimensi hanyalah $(\mathbf{A} \odot \mathbf{B})_{ij} = A_{ij}B_{ij}$. Sebagai informasi, di dalam ilmu matematika, operasi ini dikenal sebagai hasil kali Hadamard. Operasi perkalian matriks yang kita pelajari dari aljabar linear mengikuti rumus yang berbeda, yakni $(\mathbf{AB})_{ij} = \sum_k A_{ik}B_{kj}$. Operasi ini secara rapi sudah ada di dalam fungsi `dot()` milik NumPy:

```
np.dot(A,B)
```

Output:

```
array([[ 90,  96, 102],
       [216, 231, 246],
       [342, 366, 390]])
```

Hasilnya sesuai yang diharapkan untuk suatu perkalian matriks biasa, tetapi sintaksisnya mungkin terasa kurang intuitif. Untungnya, sejak Python 3.5 dan seterusnya kita dapat mengalikan dua buah matriks menggunakan operator *infix* `@`, yaitu:

```
A@B
```

Output:

```
array([[ 90,  96, 102],
       [216, 231, 246],
       [342, 366, 390]])
```

yang jelas jauh lebih enak digunakan. Uniknya, baik `@` maupun `numpy.dot()` mampu menangani larik 1D.

NumPy masih memiliki beberapa fungsi penting lain, dengan nama yang mendeskripsikan kegunaannya sendiri secara gamblang. Yang paling umum digunakan adalah `transpose()`:

```
np.transpose(A)
```

Output:

```
array([[1, 4, 7],
       [2, 5, 8],
       [3, 6, 9]])
```

yang juga dapat dipanggil sebagai atribut dari larik:

```
A.T
```

Output:

```
array([[1, 4, 7],
       [2, 5, 8],
       [3, 6, 9]])
```

Fungsi lain yang sangat berguna adalah `trace()`:

```
np.trace(A)
```

Output:

```
15
```

Seperti yang mungkin sudah kita tebak, fungsi terakhir ini akan menjumlahkan elemen-elemen yang berada pada diagonal utama matriks.

Berdasarkan hal-hal yang telah kita perkenalkan, kita dapat melihat betapa mudahnya mengiterasi baris-baris pada sebuah larik:

```
A = np.arange(1,10).reshape(3,3)
A
```

Output:

```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

```
for row in A:
    print(row)
```

Output:

```
[1 2 3]
[4 5 6]
[7 8 9]
```

maupun kolom-kolomnya:

```
for column in A.T:
    print(column)
```

Output:

```
[1 4 7]
[2 5 8]
[3 6 9]
```

Perhatikan bahwa dalam kedua kasus iterasi di atas kita tidak melihat adanya kurung siku ganda, karena kita mencetak sebuah larik 1D di setiap iterasinya. Iterasi melalui kolom akan sangat berguna terutama ketika kita menangani nilai *eigen* (dan *eigenvectors*) dari sebuah matriks.

Secara umum, larik 2D memiliki perilaku yang sangat mirip dengan larik 1D. Artinya, saat kita menggabungkan larik 2D dengan suatu nilai skalar, proses *broadcasting* akan terjadi, contohnya:

```
A/2
```

Output:

```
array([[0.5, 1. , 1.5],
       [2. , 2.5, 3. ],
       [3.5, 4. , 4.5]])
```

Broadcasting sering kali dimanfaatkan dalam skenario yang lebih kompleks (misalnya, untuk menggabungkan array dua dimensi dengan array satu dimensi), tetapi kita tidak akan menggunakan fungsionalitas semacam itu. Tentu saja, semua fungsi NumPy yang pernah kita singgung sebelumnya, seperti `numpy.sqrt()`, `numpy.exp()`, juga bisa dijalankan pada larik 2D, dan prinsip yang sama berlaku untuk proses pengurangan, perkalian, perpangkatan, dsb.

Menariknya, fungsi-fungsi seperti `numpy.sum()`, `numpy.amin()`, dan `numpy.amax()` masih mau menerima argumen (opsional) berupa `axis`. Jika kita menetapkan `axis = 0`, operasi akan berjalan di sepanjang baris, sementara untuk `axis = 1` beroperasi di sepanjang kolom. Sebagai contoh:

```
A = np.arange(1,10).reshape(3,3)
np.sum(A,axis=0)
```

Output:

```
array([12, 15, 18])
```

Serupa dengan itu, kita dapat menyusun fungsi-fungsi bebas buatan kita sendiri yang dirancang untuk sanggup menangani matriks 2D seutuhnya secara bersamaan. Mengingat fitur-fitur ini sangat identik dengan apa yang kita temui pada bagian

sebelumnya, dapat diasumsikan konsepnya sudah jelas bagi pembaca. Meskipun begitu, memang ada beberapa fungsi praktis yang secara intuitif lebih mudah dicerna penggunaannya untuk larik 2D seperti `numpy.diag()` yang digunakan untuk mendapatkan elemen-elemen dari diagonal, atau `numpy.tril()` untuk memperoleh segitiga bagian bawah (*lower triangle*) dari sebuah array serta `numpy.triu()` untuk segitiga bagian atas (*upper triangle*).

Terakhir, sebuah fungsi berguna lainnya adalah `numpy.eye()` yang wujudnya sangat ekuivalen dengan `numpy.identity()`. Namun, yang membuatnya lebih praktis, fungsi ini mengizinkan kita mengisi elemen pada diagonal yang letaknya ada di atas atau di bawah diagonal utama. Sebagai contoh, cobalah eksekusi `numpy.eye(5, k=-2)`. Kita dapat menelaah kembali dokumentasi yang ada untuk menggali lebih lanjut rupa-rupa fungsionalitas ini.

Satu poin tambahan, format penyimpanan bawaan untuk larik NumPy di dalam memori komputer adalah *row-major* (mengutamakan baris). Sebagai seorang pemula, kita tidak perlu terlalu merisaukan perihal ini, tetapi artinya adalah baris-baris akan disimpan secara berurutan, satu demi satu. Ini adalah format penyimpanan serupa yang diadopsi oleh bahasa pemrograman C. Jika sebaliknya kita ingin menggunakan format *column-major* (mengutamakan kolom) ala Fortran, mungkin demi tujuan bisa berinteraksi dengan kode dari bahasa tersebut, kita hanya perlu meneruskan argumen *keyword* yang sesuai saat membuat larik NumPy kita. Selama kita menyusun kode kita dalam cara yang "natural", yakni melakukan iterasi melalui baris terlebih dahulu dan barulah melalui kolom, segalanya akan berjalan normal dan mulus, sehingga kita sebenarnya dapat saja mengabaikan paragraf ini.

Studi Kasus

Kasus 1

Buatlah sebuah program Python dengan NumPy untuk melakukan operasi berikut pada sebuah matriks A berukuran 2×3:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}.$$

Lakukan langkah-langkah berikut:

1. Tampilkan *shape* dari matriks A.
2. Bentuk *transpose* dari A, yaitu A^T.
3. Hitung hasil perkalian matriks $B = AA^T$.

Solusi

Kita mulai dengan membentuk matriks A sebagai larik NumPy 2D. Setelah itu, *shape* matriks dapat diperoleh melalui atribut `shape`. Transpose matriks diperoleh dengan `A.T`. Untuk perkalian matriks, kita gunakan operator `@`, karena yang diminta adalah perkalian matriks, bukan perkalian elemen per elemen.

```python
import numpy as np

# Membentuk matriks A
A = np.array([[1, 2, 3],
              [4, 5, 6]])

# Shape matriks
shape_A = A.shape

# Transpose matriks
AT = A.T

# Perkalian matriks A dengan transpose-nya
B = A @ AT

# Menampilkan hasil
print("A =")
print(A)
print("shape(A) =", shape_A)
print("A^T =")
print(AT)
print("B = A @ A^T =")
print(B)
```

Output:

```
A =
[[1 2 3]
 [4 5 6]]
shape(A) = (2, 3)
A^T =
[[1 4]
 [2 5]
 [3 6]]
B = A @ A^T =
[[14 32]
 [32 77]]
```

Sekarang kita jelaskan hasilnya. Matriks

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

memiliki 2 baris dan 3 kolom, sehingga

$$\text{shape}(A) = (2, 3).$$

Transpose matriks diperoleh dengan menukar baris menjadi kolom, sehingga

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}.$$

Selanjutnya kita hitung

$$B = AA^T.$$

Karena A berukuran 2×3 dan A^T berukuran 3×2, hasil perkalian akan berukuran 2×2. Perhitungannya adalah

$$B = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}.$$

Elemen-elemen matriks hasil diperoleh dari hasil kali titik baris dengan kolom:

$$B_{11} = 1 \cdot 1 + 2 \cdot 2 + 3 \cdot 3 = 14,$$

$$B_{12} = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 32,$$

$$B_{21} = 4 \cdot 1 + 5 \cdot 2 + 6 \cdot 3 = 32,$$

$$B_{22} = 4 \cdot 4 + 5 \cdot 5 + 6 \cdot 6 = 77.$$

Jadi,

$$B = \begin{bmatrix} 14 & 32 \\ 32 & 77 \end{bmatrix}.$$

Kasus 2

Buatlah sebuah program Python dengan NumPy untuk mempelajari indexing, slicing, dan operasi sepanjang sumbu tertentu pada sebuah matriks. Diberikan

matriks

$$M = \begin{bmatrix} 2 & 4 & 6 \\ 1 & 3 & 5 \\ 7 & 8 & 9 \end{bmatrix}.$$

Lakukan langkah-langkah berikut: (1) Ambil baris kedua dari matriks M, (2) Ambil kolom ketiga dari matriks M, (3) Ambil submatriks yang terdiri atas dua baris pertama dan dua kolom pertama, (4) Hitung jumlah setiap baris, dan (5) Hitung jumlah setiap kolom.

Solusi

Pada soal ini, kita akan menggunakan *indexing* dan *slicing* pada larik 2D. Dalam NumPy, elemen diakses dengan format `[baris, kolom]`. Untuk mengambil satu baris atau satu kolom, kita dapat menggunakan *slicing* yang sesuai. Penjumlahan sepanjang baris atau kolom dilakukan dengan fungsi `np.sum()` menggunakan parameter `axis`.

```python
import numpy as np

# Membentuk matriks M
M = np.array([[2, 4, 6],
              [1, 3, 5],
              [7, 8, 9]])

# Mengambil bagian-bagian matriks
baris_kedua = M[1, :]
kolom_ketiga = M[:, 2]
submatriks = M[:2, :2]

# Menjumlahkan elemen per baris dan per kolom
jumlah_baris = np.sum(M, axis=1)
jumlah_kolom = np.sum(M, axis=0)

# Menampilkan hasil
print("M =")
print(M)
print("Baris kedua =", baris_kedua)
print("Kolom ketiga =", kolom_ketiga)
print("Submatriks dua baris pertama dan dua kolom pertama =")
print(submatriks)
print("Jumlah setiap baris =", jumlah_baris)
print("Jumlah setiap kolom =", jumlah_kolom)
```

Output:

```
M =
[[2 4 6]
 [1 3 5]
 [7 8 9]]
Baris kedua = [1 3 5]
Kolom ketiga = [6 5 9]
Submatriks dua baris pertama dan dua kolom pertama =
[[2 4]
 [1 3]]
Jumlah setiap baris = [12  9 24]
Jumlah setiap kolom = [10 15 20]
```

Sekarang kita bahas satu per satu. Matriks yang diberikan adalah

$$M = \begin{bmatrix} 2 & 4 & 6 \\ 1 & 3 & 5 \\ 7 & 8 & 9 \end{bmatrix}.$$

Baris kedua diambil dengan

`M[1, :]`

karena indeks Python dimulai dari 0. Jadi, baris kedua adalah

$$[1, 3, 5].$$

Kolom ketiga diambil dengan

`M[:, 2],`

artinya semua baris diambil, tetapi hanya kolom dengan indeks 2. Hasilnya adalah

$$[6, 5, 9].$$

Submatriks yang terdiri atas dua baris pertama dan dua kolom pertama diambil dengan

`M[:2, :2],`

sehingga diperoleh

$$\begin{bmatrix} 2 & 4 \\ 1 & 3 \end{bmatrix}.$$

Selanjutnya, jumlah setiap baris dihitung dengan

`np.sum(M, axis=1).`

Artinya, penjumlahan dilakukan sepanjang kolom-kolom pada setiap baris. Hasilnya:

$$2 + 4 + 6 = 12,$$

$$1 + 3 + 5 = 9,$$

$$7 + 8 + 9 = 24.$$

Jadi, kita peroleh jumlah setiap baris adalah:

$$[12, 9, 24].$$

Jumlah setiap kolom dihitung dengan

```
np.sum(M, axis=0).
```

Artinya, penjumlahan dilakukan sepanjang baris-baris pada setiap kolom. Hasilnya:

$$2 + 1 + 7 = 10,$$

$$4 + 3 + 8 = 15,$$

$$6 + 5 + 9 = 20.$$

Jadi, kita peroleh jumlah setiap kolom adalah:

$$[10, 15, 20].$$